

Detailed Software Design Description

Sammy's Games

6.1 Introduction

This document presents the architecture and detailed software design for the Sammy's Games project. Sammy's Games is a browser-based gaming platform implementing original strategy and card games, with the current deliverable centered on **Knockout Bot** — a two-player abstract strategy game with a computer-controlled AI opponent — and **Six Hands**, a daily puzzle game based on poker hand evaluation.

The purpose of this document is to establish a shared architectural understanding sufficient to guide implementation, testing, and future expansion. It covers the major software components, their interactions, class-level designs, data structures, and database considerations. All design descriptions are grounded in the existing JavaScript source code for the platform.

6.1.1 System Objectives

The primary objectives of the Sammy's Games platform are as follows:

- Deliver 2–3 fully functional, browser-based strategy and puzzle games within a single semester, beginning with Knockout Bot and Six Hands.
- Enforce all gameplay rules completely and correctly so that invalid moves are never accepted and win conditions are reliably detected.
- Implement an AI opponent capable of selecting legal, non-self-destructive moves and recognizing immediate winning opportunities via single-move lookahead simulation.
- Provide a clean, intuitive graphical user interface that communicates game state clearly through visual highlighting, move history, and win announcements.
- Maintain a modular architecture that makes it straightforward to add new games to the platform without redesigning existing components.
- Ensure the system is maintainable and readable, with clearly separated responsibilities across logical subsystems to support solo and collaborative development.
- Support a responsive layout that renders correctly across modern desktop browsers (Chrome, Firefox, Safari, Edge) at standard resolutions of 1024×768 and above.

The Six Hands puzzle additionally aims to generate a deterministic daily puzzle based on a date-derived seed, ensuring all users worldwide play the same puzzle each day.

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Hardware Interface

The application operates on any standard personal computing hardware meeting the following minimum requirements:

- A display monitor with a minimum resolution of 1024×768 pixels. The board layouts and card grids are designed for desktop viewing; very small displays may require horizontal scrolling.
- A pointing device (mouse or trackpad) for all gameplay interactions. All game actions are click-based.
- A keyboard is present on standard hardware but is not currently required for gameplay.
- An internet connection is required only if the application is accessed from a remote host. When run locally, no network access is needed.

No specialized hardware (graphics acceleration, game controllers, touch screens, etc.) is required.

6.1.2.2 Software Interface

The system is built entirely with standard web technologies and requires no installation by the end user:

- **HTML5** – provides document structure, the game board container (div#board), the card grid (div.grid), navigation, and rules panels.
- **CSS3 with Grid Layout** – manages visual arrangement of the 4×4 Knockout board (grid-template-columns: repeat(4, 80px)) and the 6×5 Six Hands card grid. Flexbox is used for page-level layout.
- **Vanilla JavaScript (ES6+)** – implements all game logic, AI decision-making, state management, event handling, and DOM manipulation. No external libraries or frameworks are used.
- **Modern web browser** – Chrome 100+, Firefox 100+, Safari 15+, or Edge 100+ are supported. The application relies on standard DOM APIs, CSS Grid, ES6 array methods, and dataset attributes.

The application is entirely client-side. There is no server, no backend, no database, and no network calls during gameplay. All state is held in JavaScript memory for the duration of each session.

6.1.2.3 Human Interface (Graphical User Interface)

Knockout Bot Interface:

- A 4×4 game board rendered as a CSS Grid. Each cell is 80×80 px with a 5 px gap. Cells contain 'X', 'O', or are empty.
- A rules panel on the left explains movement and push mechanics in plain text.

- A move information panel to the right of the board displays the last move direction for each player (e.g., "↑ Up", "← Left").
- A win announcement banner (crimson, centered, absolute-positioned) appears when a piece is pushed off the board, ending all interactivity.
- User interaction flow: (1) click an owned piece to select it (cell turns yellow); (2) valid destinations turn sky-blue; (3) click a valid destination to execute the move; (4) the AI responds automatically after a 1-second delay.

Six Hands Interface:

- A 6×5 grid of 30 styled playing cards rendered as HTML div elements. Each card displays rank, suit symbol, and color coding (red = hearts, blue = diamonds, green = clubs, black = spades).
- A color picker row above the grid allows selection of one of five highlight colors (yellow, green, blue, pink, purple) or a star selector for the sixth hand.
- A rules panel on the left explains grouping and hand-selection mechanics.
- A hands panel on the right shows each group's evaluated poker hand name, the sixth hand, and a validity indicator (green ✓ or red ✗).
- The lowest-scoring valid hand is highlighted in gold with a star marker as the puzzle score.

6.2 Architectural Design

Sammy's Games follows a modular, single-page application architecture implemented entirely in client-side JavaScript. Although the codebase does not use a formal framework, it conceptually separates responsibilities into five distinct subsystems that mirror the classic Model-View-Controller (MVC) pattern. This separation reduces coupling, clarifies each component's responsibilities, and makes it practical to add new games without restructuring existing code.

State flows in one direction: user events enter through the Event Control subsystem, which delegates to Game Logic for validation, updates State Management as the authoritative data store, and finally triggers the UI Renderer to reflect the new state. The AI subsystem operates as a special actor that triggers moves through the same Event Control path used by the human player, ensuring consistent validation.

6.2.1 Major Software Components

1. State Management Subsystem – Central data store for all live game data. For Knockout Bot this includes: boardState (16-element array, null/'X'/'O'), xturn (boolean), selectedIndex(integer|null), lastMoveX (string), and lastMoveO (string). For Six Hands this includes per-card highlight color and star-selection state stored as HTML dataset attributes on each card element.

2. Game Logic Subsystem – Rule enforcement engine. For Knockout: validMove(start, end) checks row/column alignment, path obstruction, adjacency for push moves, and the

repeated-direction constraint. `moveDirection(start, end)` maps index pairs to compass labels. Push-chain resolution walks the push path iteratively, detecting edge-knockout and cascade pushes. For Six Hands: `isConnected(cards, color)` performs BFS on the 6×5 grid to verify orthogonal connectivity of each six-card group. `evaluateHand(cards)` scores five-card hands including joker substitution.

3. AI Decision Subsystem – Computer player for Knockout Bot. `botMove()` enumerates all legal moves for 'X' pieces, clones `boardState` via `spread`, runs `simulateMove()` on the clone, and classifies each outcome. Winning moves (reducing O count to 3) are played immediately. If no win exists, a move that preserves all four X pieces is selected at random from the safe-moves list.

4. User Interface Rendering Subsystem – DOM update layer. `updateBoard()` iterates all 16 cells, synchronizes `textContent` with `boardState`, applies 'selected' and 'valid' CSS classes, and calls `announce()` for the win banner. The Six Hands equivalent (`updateGroupsAndValidation()`) rebuilds the groups panel, re-evaluates all six hands, checks validity constraints, and marks the lowest hand.

5. Event Control Subsystem – Controller and game-flow coordinator. `handleClick(i)` manages the selection-move state machine: deselects if same cell clicked, executes a move if a valid destination is chosen (invoking push-chain logic and then triggering `botMove` via `setTimeout`), or selects a piece belonging to the current player. Guards prevent interaction after game-over.

6.2.2 Major Software Interactions

All subsystem interactions are synchronous function calls within a single JavaScript execution context. There are no network requests, Web Workers, or asynchronous operations other than the 1-second `setTimeout` used to give the AI response a natural feel.

Knockout Bot – normal move sequence:

1. User clicks a board cell. `handleClick(i)` is invoked by the cell's onclick handler.
2. If a piece is already selected, `handleClick` calls `validMove(selectedIndex, i)`. Invalid destination: nothing happens.
3. Valid destination with empty target: `boardState` is updated, `xturn` toggles, `selectedIndex` clears.
4. `updateBoard()` refreshes all cell `textContent` and CSS classes.
5. `setTimeout(botMove, 1000)` schedules the AI turn.

Knockout Bot – push move sequence (same as above through step 2, then):

1. Target cell is occupied. Push direction is derived via `moveDirection()`. The moving piece is placed, and a while-loop walks the push chain: if the pushed piece's current position is an edge cell, `announce()` is called and the game ends; otherwise the piece advances one step, displacing any piece already there.
2. `updateBoard()` is called after each displacement to animate the chain visually.
3. After push resolution, `botMove` is scheduled via `setTimeout`.

AI turn sequence:

1. botMove() iterates all 16 cells, testing each 'X' piece against all 16 possible destinations via validMove().
2. Each legal move is simulated on a cloned board via simulateMove(testBoard, start, end).
3. Post-simulation O-count and X-count determine move classification (winning vs. safe).
4. The selected move executes by setting selectedIndex and calling handleClick(end), routing back through the same validation and update logic as a human move.

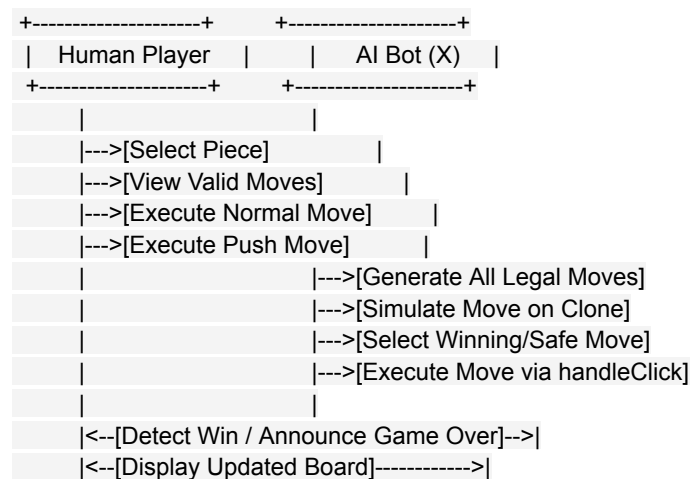
Error / invalid-move handling: When validMove() returns false, handleClick() silently takes no action and does not update the board. The selected piece remains highlighted and the user may try a different destination. After game-over, the announce() banner overlays the board and the xturn flag is no longer toggled, so further input is harmlessly ignored.

Concurrency considerations: Because JavaScript is single-threaded, no true concurrency exists. The 1-second AI delay is cosmetic. During that window the human player's clicks are silently ignored because xturn has already toggled to 'X' and clicks on 'O' pieces fail the ownership check in handleClick(). This is a deterministic guard, not a lock.

6.2.3 Architectural Design Diagrams

Figure 1 – Use Case Diagram

Use Case Diagram – Knockout Bot



Use Case Diagram – Six Hands



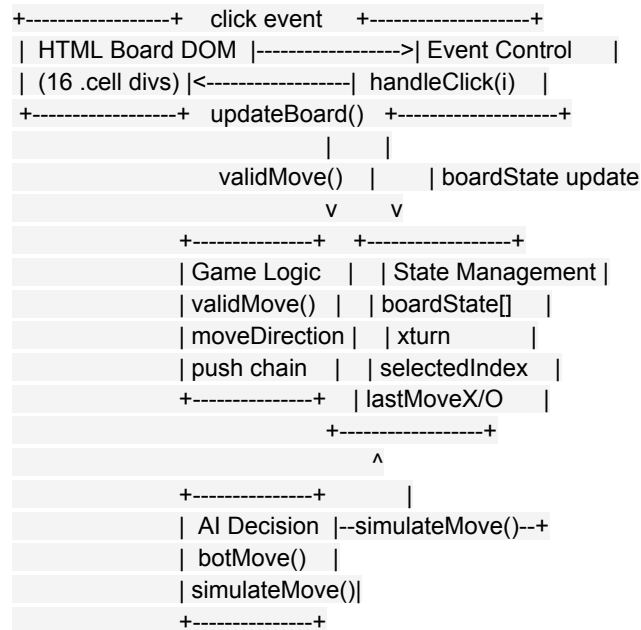
```

|--->[Assign Card to Color Group]
|--->[Remove Card from Color Group]
|--->[Select Star (6th Hand) Card]
|--->[View Hand Evaluations]
|--->[View Validation Status]
|--->[View Lowest-Hand Score]

```

Figure 2 – Top-Level Component/Class Diagram

Component Diagram – Knockout Bot



Component Diagram – Six Hands

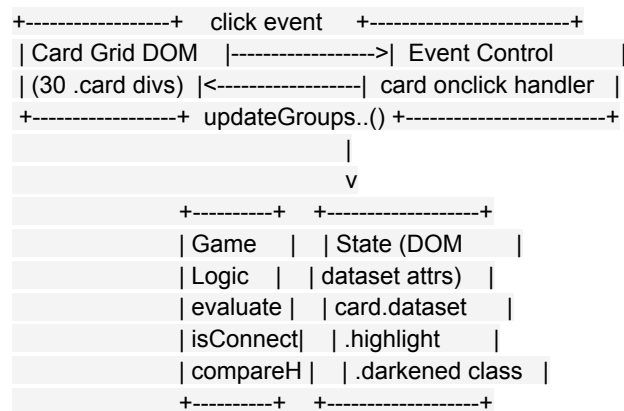


Figure 3 – Sequence Diagram: Full Human + Bot Turn

Sequence Diagram – One Full Turn Cycle (Knockout Bot)

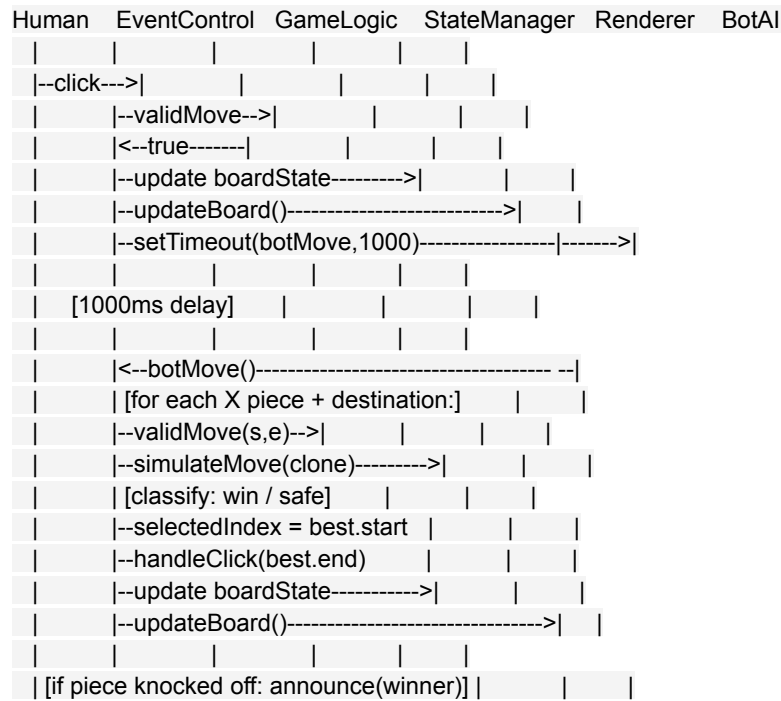
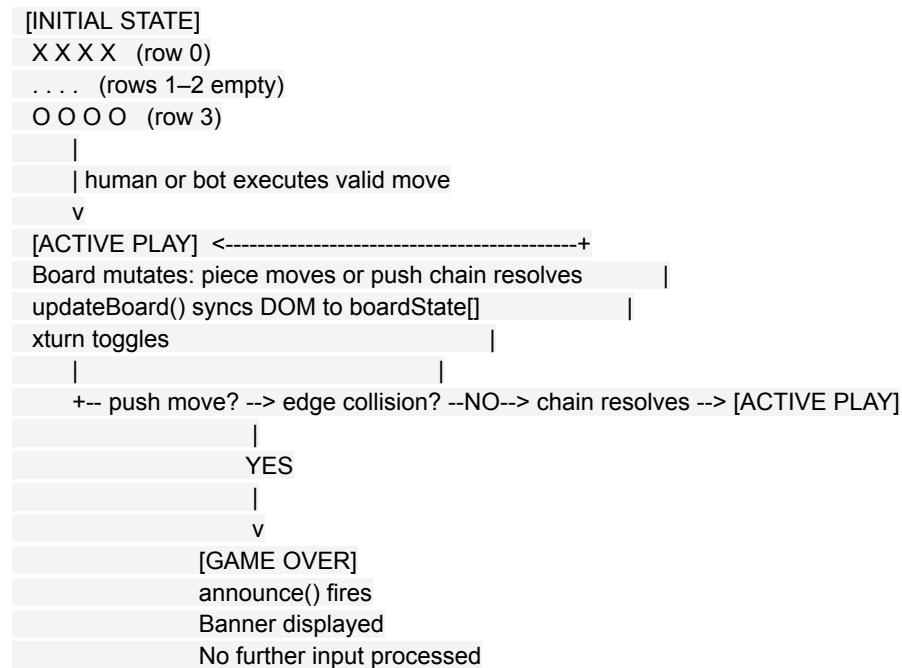


Figure 4 – State Transition Diagram: Game Board

State Transition Diagram – Knockout Bot Board State



6.3 CSC and CSU Descriptions

The Sammy's Games platform (the CSCI) is organized into two Computer Software Components (CSCs): the **Knockout Bot CSC** and the **Six Hands CSC**. Each CSC corresponds to one game and contains its own Computer Software Units (CSUs) organized by functional responsibility. A shared navigation and index module connects them at the presentation layer.

6.3.1 Detailed Class Descriptions

The following sections describe all logical classes in the Sammy's Games application. Because the codebase uses vanilla JavaScript (no class keyword), each "class" is represented as a cohesive set of functions and variables sharing a common concern. They are described here in class terms to satisfy the SDD standard and to clarify boundaries for any future refactoring into formal classes or modules.

6.3.1.1 BoardStateManager (Knockout Bot CSC)

This logical class owns all mutable state for a Knockout Bot game session. It is implemented as module-level variables in knockout_bot.html. Its state is read by every other subsystem and written exclusively through the Event Control flow to prevent accidental corruption.

Fields		
Field Name	Type	Description
boardState	Array[16]	One-dimensional array representing the 4×4 grid in row-major order. Index 0 = top-left, index 15 = bottom-right. Values: null (empty), 'X', or 'O'.
selectedIndex	integer null	Index of the currently selected cell, or null if no piece is selected.
xturn	boolean	True when it is X's turn (AI), false when it is O's turn (human player).
lastMoveX	string	Arrow-label of X's most recent move direction (e.g., "↑ Up"). Initialized to "None".
lastMoveO	string	Arrow-label of O's most recent move direction. Initialized to "None". Used to enforce the no-repeat-direction rule.
Methods		

Method Name	Returns	Description
(initialization)	void	Sets boardState[0..3] = 'X', boardState[12..15] = 'O', all others null. Randomly selects first turn. If X starts, calls setTimeout(botMove, 1000).

6.3.1.2 MoveValidator (Knockout Bot CSC)

This logical class encapsulates all move-legality checks. It is stateless; it reads boardState and current turn state but never modifies them.

Fields		
Field Name	Type	Description
<i>(none — stateless)</i>		
Methods		
Method Name	Returns	Description
validMove(start, end)	boolean	Returns true if the move from index start to index end is legal given the current board and turn state. Checks same-row/column alignment, path clearance for normal moves, adjacency and no-repeat constraints for push moves, and endpoint vacancy for normal moves.
moveDirection(start, end)	string	Returns a human-readable arrow string ("↑ Up", "↓ Down", "← Left", "→ Right") for the direction of travel from start to end.

6.3.1.3 PushEngine (Knockout Bot CSC)

This logical class handles the execution of push moves and their cascading chain effects. It is invoked by the Event Control subsystem when a validated push move is to be executed on the live board.

Fields		
Field Name	Type	Description
step	integer	Direction offset for the push chain: -4 (up), +4 (down), -1 (left), +1 (right). Set per-move inside handleClick().
edges	Array[4]	Set of edge indices for the push direction. If the pushed piece is on one of these before advancing, it falls off the board.
pushedPiece	string	The piece ('X' or 'O') currently being displaced through the push chain.
pushedIndex	integer	Current board index of the piece being displaced.
Methods		
Method Name	Returns	Description
(push chain loop)	void	Embedded while-loop in handleClick(). Checks edge condition; if on edge, calls announce(). Otherwise advances pushedIndex by step, placing pushedPiece. If target occupied, cascades by saving displaced piece and repeating. Calls updateBoard() after each placement.
announce(message)	void	Makes the #announcement div visible with the given message string. Called when a piece is pushed off the board.

6.3.1.4 EventController (Knockout Bot CSC)

This is the central controller class. It wires DOM events to game logic and coordinates the turn cycle. All player interaction enters the system through this class.

Fields		
Field Name	Type	Description
<i>(none — stateless controller)</i>		
Methods		
Method Name	Returns	Description
handleClick(i)	void	Primary event handler called on any cell click with the cell's board index i. (1) Deselects if i equals selectedIndex. (2) If selectedIndex is set and validMove returns true, executes the move (normal or push), toggles xturn, schedules botMove if xturn becomes true, calls updateBoard(). (3) Otherwise, selects the piece at i if it belongs to the current player.

6.3.1.5 BotAI (Knockout Bot CSC)

This class implements the computer opponent for Knockout Bot. It operates on a cloned board to simulate moves without affecting live state, then executes its chosen move through the standard EventController path.

Fields		
Field Name	Type	Description
safeMoves	Array<{start,end}>	Accumulates all legal moves that preserve all four X pieces after simulation.
winningMove	{start,end} null	Set to the first discovered move that reduces the O count to 3 (knocks out an opponent piece).
Methods		

Method Name	Returns	Description
botMove()	void	Enumerates all X pieces, tests all 16 destinations via validMove(), simulates each legal move on a clone via simulateMove(), classifies as winning or safe, then executes the best found move by setting selectedIndex and calling handleClick().
simulateMove(testBoard , start, end)	void	Mirrors the live push-chain logic on a provided board array. Handles both normal moves and push chains including piece-falls-off-board. Mutates testBoard in-place; the clone is disposable.

6.3.1.6 BoardRenderer (Knockout Bot CSC)

This class is responsible for keeping the visual DOM representation synchronized with boardState. It has no state of its own and operates purely as a rendering pass.

Fields		
Field Name	Type	Description
<i>(none — stateless)</i>		
Methods		
Method Name	Returns	Description
updateBoard()	void	Iterates all 16 .cell elements. Sets textContent to boardState[i]. Applies 'valid' CSS class if validMove(selectedIndex, i) returns true. Applies 'selected' CSS class if i equals selectedIndex. Removes stale classes otherwise.
announce(message)	void	Sets #announcement div textContent to message and sets its display to 'block'. Banner remains visible for the rest of the session.

6.3.1.7 SixHandsStateManager (Six Hands CSC)

Manages persistent puzzle state for the Six Hands game. Unlike Knockout Bot, this class stores state in DOM dataset attributes on card elements rather than in a separate array, making each card element the authoritative record of its own state.

Fields		
Field Name	Type	Description
currentHighlight	string (CSS color)	The currently selected highlight color code, or the blackMode sentinel ('#000000') for sixth-hand selection. Updated by the color picker buttons.
highlightColors	Array<{code,label}>	Static array of the five group colors with their CSS hex codes and display labels. Determines the order of buttons in the color picker.
blackMode	string	Sentinel value '#000000' indicating star/sixth-hand selection mode is active.
Methods		
Method Name	Returns	Description
getDailySeed()	integer	Computes a deterministic integer seed from today's UTC date string using a polynomial hash. Produces the same seed worldwide on any given calendar day.
mulberry32(seed)	function	Returns a seeded pseudo-random number generator closure using the mulberry32 algorithm. Used to shuffle the deck reproducibly from the daily seed.
createDeck()	Array<string>	Builds and returns a 28-element array: 26 suited cards (rank+suit strings) plus two joker tokens ("JOKER", "joker").

shuffle(deck, random)	void	Fisher-Yates shuffle using the provided seeded RNG. Mutates the deck array in-place.
dealGrid()	void	Generates today's deck, shuffles it, creates 30 card div elements with inline HTML for rank/suit display, attaches click handlers, and appends them to #cardGrid.

6.3.1.8 HandEvaluator (Six Hands CSC)

Provides the poker hand evaluation logic used to score each of the six groups. Supports joker substitution by trying every possible replacement card and returning the best achievable score.

Fields		
Field Name	Type	Description
<i>(none — stateless)</i>		
Methods		
Method Name	Returns	Description
evaluateHand(cards)	{score, name, rankVector }	Entry point. If no joker present, delegates directly to scoreHand(). If a joker is present, iterates all valid replacement candidates, calls scoreHand() on each, and returns the result with the maximum rankVector.
scoreHand(hand)	{score, name, rankVector }	Scores a concrete five-card hand (no jokers). Parses ranks and suits, builds frequency groups, detects flush and straight (including wheel A-2-3-4-5). Returns a score integer (1–10), a name string, and a rankVector for comparison.
compareHands(a, b)	integer	Lexicographic comparison of two rankVector arrays. Returns positive if a > b, negative if a < b, 0 if equal. Used to find the minimum hand and for joker optimization.

6.3.1.9 GroupValidator (Six Hands CSC)

Validates that all five colored groups are well-formed before scoring is reported. Enforces the connectivity and count constraints that define a valid puzzle solution.

Fields		
Field Name	Type	Description
<i>(none — stateless)</i>		
Methods		
Method Name	Returns	Description
isConnected(cards, color)	boolean	BFS on the 6×5 card grid. Collects all card-element indices with the given highlight color, checks that exactly 6 exist, then verifies all 6 are reachable from the first via orthogonal adjacency. Returns true only if both constraints pass.
updateGroupsAndValidation() ()	void	Master update function for Six Hands UI. Recomputes group membership from DOM dataset attributes, evaluates each hand, checks all validity constraints (allColored, sixPerColor, oneDarkEach, fiveDarkTotal, connectedGroups), updates the hands panel, shows/hides the validity indicator, and marks the lowest hand in gold.

6.3.2 Detailed Interface Descriptions

EventControl → GameLogic interface: The EventController calls validMove(start, end) and moveDirection(start, end) as synchronous function calls. Both functions accept two integer indices and return immediately. They read the module-level boardState array directly; no mutable state is passed by reference.

EventControl → StateManager interface: The controller reads and writes boardState[], xturn, selectedIndex, lastMoveX, and lastMoveO directly as module-level variables. All subsystems in the same file share the same scope. Future modularization would introduce formal accessors.

EventControl → Renderer interface: The controller calls `updateBoard()` after any state change. `updateBoard()` takes no parameters and reads all needed state from the shared scope. The only data passed explicitly is the message string to `announce()`.

EventControl → BotAI interface: The controller schedules `botMove()` via `setTimeout(botMove, 1000)` after a human move completes. This is the only asynchronous interface in the system. `botMove()` re-enters the system by calling `handleClick()` directly, completing the full validation-update-render cycle.

BotAI → GameLogic interface: `botMove()` calls `validMove()` for each candidate move and calls `simulateMove()` on a cloned board. `simulateMove()` accepts a board array by reference and mutates it; this is intentional since the clone is disposable.

Six Hands DOM-as-state interface: Six Hands stores all group assignment and sixth-hand selection state as HTML dataset attributes (`card.dataset.highlight`, `card.classList 'darkened'`) on the 30 card elements. `updateGroupsAndValidation()` reads these attributes via `querySelectorAll` and dataset access. This avoids a separate state array but couples state to the DOM.

6.3.3 Detailed Data Structure Descriptions

boardState[] – Knockout Board Array: A flat 16-element JavaScript array mapping to the 4×4 grid in row-major order. Index arithmetic: `row = Math.floor(i/4)`, `col = i%4`. Values are null (empty), 'X', or 'O'. Orthogonal neighbors of index `i` are: `up = i-4`, `down = i+4`, `left = i-1` (same-row guard required), `right = i+1` (same-row guard required). Edge sets: top `[0,1,2,3]`, bottom `[12,13,14,15]`, left `[0,4,8,12]`, right `[3,7,11,15]`.

rankVector – Hand Comparison Vector: A variable-length array used to compare poker hands. Format: `[handScore, primaryRank, secondaryRank, ...]`. `handScore` is an integer 1–10 (1 = High Card through 10 = Royal Flush). Subsequent elements are card rank values (2–14) ordered from highest-frequency group to lowest, then by rank descending within each group. Lexicographic comparison via `compareHands()` gives a total ordering over all five-card hands. The joker substitution loop retains the maximum `rankVector` seen.

move record – AI Move Candidate: A plain JavaScript object `{ start: integer, end: integer }` representing a candidate AI move. Stored in the `safeMoves` array or the `winningMove` variable. No additional fields are required because all context (board state, direction, push chain) is re-derived from `start` and `end` at execution time.

highlightColors[] – Color Configuration Array: Static array of five objects, each `{ code: string, label: string }`. `code` is a CSS hex color used as the `dataset.highlightvalue` and as the background of the color-picker button. `label` is displayed in the groups panel. The array order determines the visual order of buttons in the color picker.

6.3.4 Detailed Design Diagrams

Figure 5 – Detailed Class Diagram (Knockout Bot)

+-----+ reads/writes +-----+

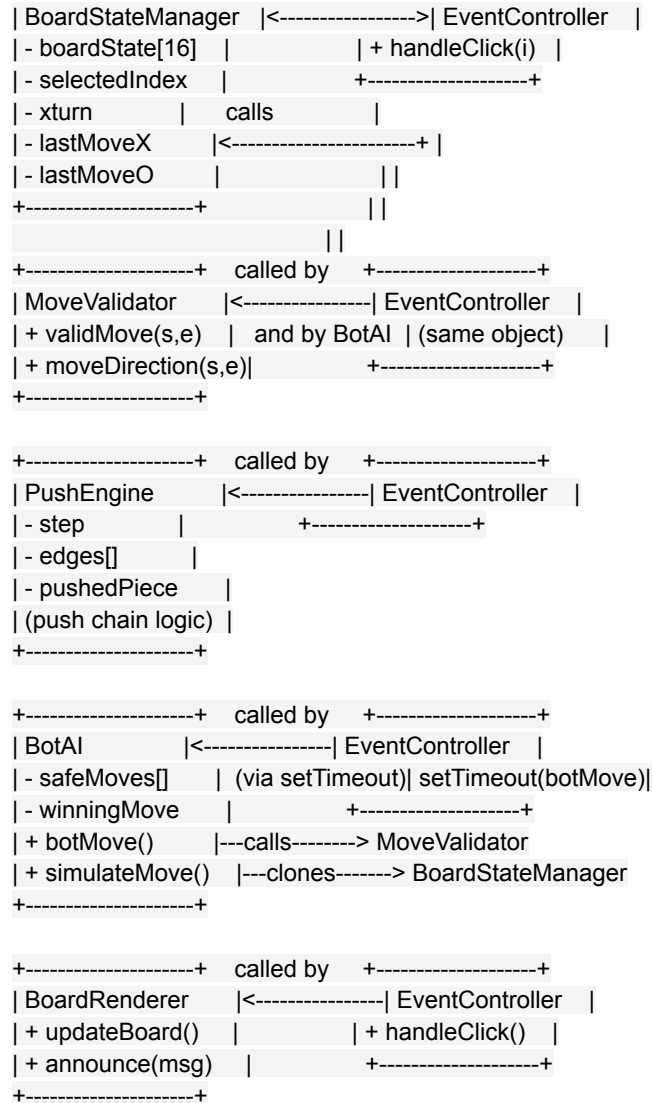
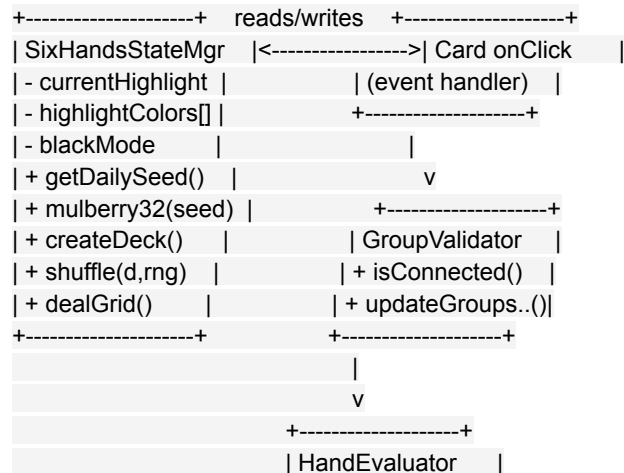


Figure 6 – Detailed Class Diagram (Six Hands)



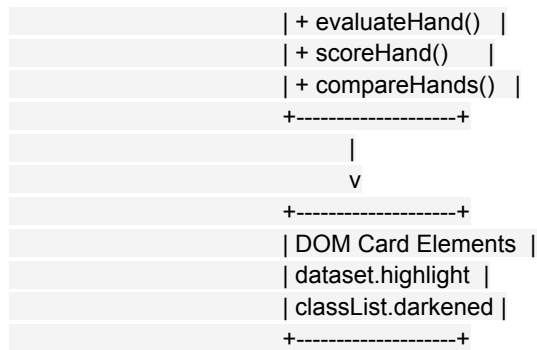
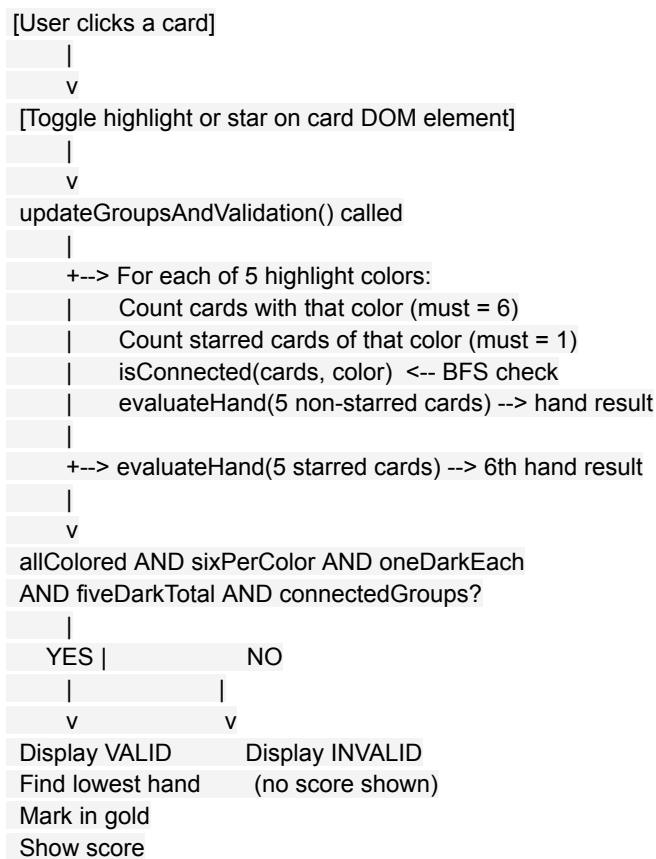


Figure 7 – Activity Diagram: Six Hands Validation Flow



6.4 Database Design and Description

Sammy's Games does not use a database in its current implementation. All game state is maintained in JavaScript memory for the duration of a browser session and is not persisted between sessions. The Six Hands puzzle derives its daily card layout deterministically from the current UTC date using the mulberry32 seeded PRNG, eliminating the need to store or retrieve puzzle configurations from any external store.

No SQL, NoSQL, or embedded database engine is used, and no network requests are made during gameplay. User scores, game history, and high scores are not currently tracked or stored.

6.4.1 Database Design ER Diagram

Not applicable. No database is used in this project. If persistence (e.g., daily scores, user accounts, leaderboards) is added in a future iteration, an ER diagram will be included covering entities such as User, PuzzleResult, and DailyPuzzle.

6.4.2 Database Access

Not applicable. See section 6.4 above.

6.4.3 Database Security

Not applicable. Since there is no database and no server-side component, there are no database credentials, query injection surfaces, or access control layers to describe. All data remains local to the user's browser tab and is discarded when the page is closed or refreshed. If a backend is introduced in a future version, security measures including parameterized queries, HTTPS transport, and session token management will be documented here.